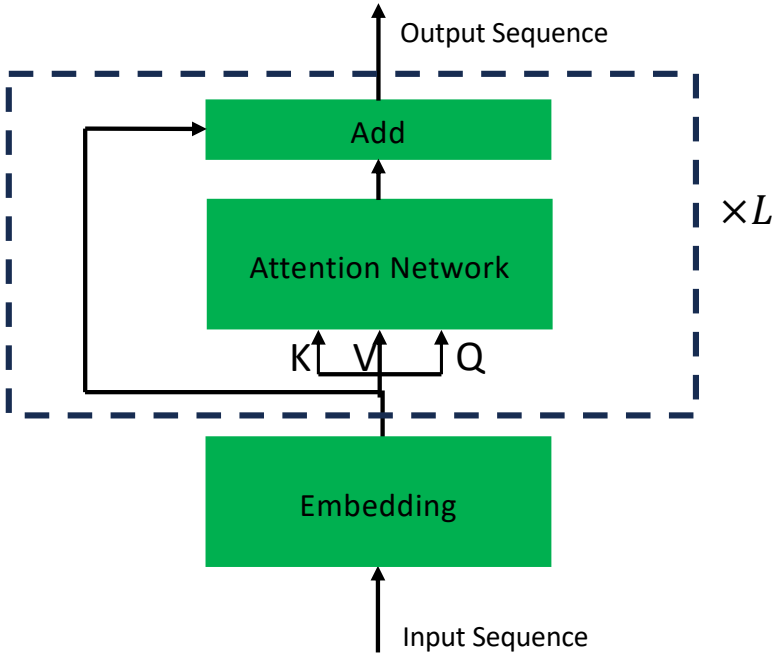
Lecture 11

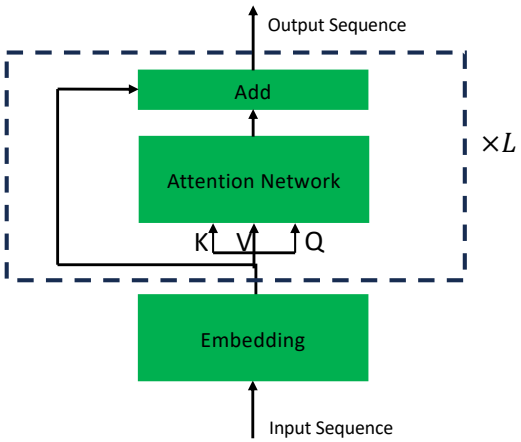
February 12, 2026

Is Attention All You Need?



- Let's isolate and try to understand one element of model: Attention plus skip connection
- What does this part of the model do?
- For now also remove positional encoding (will not be needed in problems we will examine)

Attention + Skip Connection Update Rule



- With linear attention, the update rule for layer $l = 1, \ldots, L$, plus skip connections, is now

$$e_i^{(l)} = \underbrace{e_i^{(l-1)}}_{\text{Skip Connection}} + P \sum_{j=1}^{N} \underbrace{(W_V e_j^{(l-1)})}_{\text{Values}} \underbrace{[(e_i^{(l-1)})^T e_j^{(l-1)}]}_{\text{Attention}}$$

- Have assumed, for simplicity that W_Q, W_K, W_V and P are the same at all layers (need not be so in general)

Learning to Learn: Training Data and Process

- Consider training data composed of M sets of contextual data, $\mathcal{C}_m$, for $m = 1, \ldots, M$, where

$$\mathcal{C}_m = \{(x_i^{(m)}, y_i^{(m)})\}_{i=1,N+1}$$

- Train the meta model to take in context $\{(x_i^{(m)}, y_i^{(m)})\}_{i=1,N}$ and the query $x_{N+1}^{(m)}$
- The meta model should be able to predict $y_{N+1}^{(m)}$ given the context and the query

Learning to Learn With Linear Functions

- The mth contextual dataset

$$\mathcal{C}_m = (x_1^{(m)}, y_1^{(m)}, \ldots, x_N^{(m)}, y_N^{(m)})$$

  assume a linear model, $f_{\theta_m}(x) = w_m^T x$ where $w_m \in \mathbb{R}^d$

- The latent w_m is different for each set of contextual data

- Can we train a meta model (the attention model) to take $\mathcal{C}_{M+1}$ as input, along with the query $x_{N+1}^{(M+1)}$, and to predict $y_{N+1}^{(M+1)}$ for an underlying w_{M+1} it has never seen?

- The trained model is trained such that it can perform inference based on a few examples (few-shot learning)

Functional Model Update

- Gradient descent learning of model parameters yields the parameter update rule

$$w_{k+1} = w_k + \alpha \sum_{i=1}^{N} (y_i - w_k^T x_i) x_i$$

- The associated function is updated as

$$\begin{aligned} f_{\theta_{k+1}}(x_j) &= w_k^T x_j + \alpha \sum_{i=1}^{N} (y_i - w_k^T x_i) x_j^T x_i \\ &= \underbrace{f_{\theta_k}(x_j)}_{\text{Skip Connection}} + \alpha \sum_{i=1}^{N} \underbrace{[y_i - f_{\theta_k}(x_i)]}_{\text{Values}} \underbrace{x_j^T x_i}_{\text{Attention}} \end{aligned}$$

Detailed Construction with Linear Attention

- Assume that the parameters are initialized $w_0 = 0_d$ and employ the following encoding as input to layer $k = 1$ attention:

$$\underbrace{\begin{pmatrix} x_1 \\ y_1 \end{pmatrix}}_{e_1^{(0)}} \underbrace{\begin{pmatrix} x_2 \\ y_2 \end{pmatrix}}_{e_2^{(0)}} \quad \cdots \quad \begin{pmatrix} x_N \\ y_N \end{pmatrix} \underbrace{\begin{pmatrix} x_{N+1} \\ 0 \end{pmatrix}}_{e_{N+1}^{(0)}}$$

- The update rule for linear attention is

$$e_j^{(l+1)} = e_j^{(l)} + P[W_V E_l][E_l^T \Phi e_j^{(l)}]$$

- Design Φ, W_V and P as

$$\Phi = \begin{pmatrix} I_{d\times d} & 0_{d\times 1} \\ 0_{1\times d} & 0 \end{pmatrix}, \quad W_V = \begin{pmatrix} 0_{d\times d} & 0_{d\times 1} \\ 0_{1\times d} & 1 \end{pmatrix}, \quad P = \begin{pmatrix} 0_{d\times d} & 0_{d\times 1} \\ 0_{1\times d} & -\alpha \end{pmatrix}$$

Try to Understand Layer-1 Attention & Skip Connection

$$\underbrace{\begin{pmatrix} x_1 \\ y_1 \end{pmatrix}}_{e_1^{(0)}} \underbrace{\begin{pmatrix} x_2 \\ y_2 \end{pmatrix}}_{e_2^{(0)}} \quad \cdots \quad \begin{pmatrix} x_N \\ y_N \end{pmatrix} \underbrace{\begin{pmatrix} x_{N+1} \\ 0 \end{pmatrix}}_{e_{N+1}^{(0)}}$$

- The update rule for linear attention is

$$e_j^{(1)} = e_j^{(0)} + P[W_V E_0][E_0^T \Phi e_j^{(0)}]$$

$$\Phi e_j^{(0)} = \begin{pmatrix} I_{d\times d} & 0_{d\times 1} \\ 0_{1\times d} & 0 \end{pmatrix} \begin{pmatrix} x_j \\ y_j \end{pmatrix} = \begin{pmatrix} x_j \\ 0 \end{pmatrix}$$

Try to Understand Layer-1 Attention & Skip Connection

$$\underbrace{\begin{pmatrix} x_1 \\ y_1 \end{pmatrix}}_{e_1^{(0)}} \underbrace{\begin{pmatrix} x_2 \\ y_2 \end{pmatrix}}_{e_2^{(0)}} \quad \cdots \quad \begin{pmatrix} x_N \\ y_N \end{pmatrix} \underbrace{\begin{pmatrix} x_{N+1} \\ 0 \end{pmatrix}}_{e_{N+1}^{(0)}}$$

- The update rule for linear attention is

$$e_j^{(1)} = e_j^{(0)} + P[W_V E_0][E_0^T \Phi e_j^{(0)}]$$

$$E_0^T \Phi e_j^{(0)} = E_0^T \begin{pmatrix} x_j \\ 0 \end{pmatrix} = \begin{pmatrix} x_1^T & y_1 \\ x_2^T & y_2 \\ \vdots & \vdots \\ x_{N+1}^T & 0 \end{pmatrix} \begin{pmatrix} x_j \\ 0 \end{pmatrix} = \begin{pmatrix} x_1^T x_j \\ x_2^T x_j \\ \vdots \\ x_{N+1}^T x_j \end{pmatrix}$$

Try to Understand Layer-1 Attention & Skip Connection

$$\underbrace{\begin{pmatrix} x_1 \\ y_1 \end{pmatrix}}_{e_1^{(0)}} \underbrace{\begin{pmatrix} x_2 \\ y_2 \end{pmatrix}}_{e_2^{(0)}} \quad \cdots \quad \begin{pmatrix} x_N \\ y_N \end{pmatrix} \underbrace{\begin{pmatrix} x_{N+1} \\ 0 \end{pmatrix}}_{e_{N+1}^{(0)}}$$

- The update rule for linear attention is

$$e_j^{(1)} = e_j^{(0)} + P[W_V E_0][E_0^T \Phi e_j^{(0)}]$$

$$\begin{aligned} W_V E_0 &= \begin{pmatrix} 0_{d\times d} & 0_{d\times 1} \\ 0_{1\times d} & 1 \end{pmatrix} \begin{pmatrix} x_1 & x_2 & \cdots & x_N & x_{N+1} \\ y_1 & y_2 & \cdots & y_N & 0 \end{pmatrix} \\ &= \begin{pmatrix} 0_{d\times 1} & 0_{d\times 1} & \cdots & 0_{d\times 1} & 0_{d\times 1} \\ y_1 & y_2 & \cdots & y_N & 0 \end{pmatrix} \end{aligned}$$

Try to Understand Layer-1 Attention & Skip Connection

$$\underbrace{\begin{pmatrix} x_1 \\ y_1 \end{pmatrix}}_{e_1^{(0)}} \underbrace{\begin{pmatrix} x_2 \\ y_2 \end{pmatrix}}_{e_2^{(0)}} \quad \cdots \quad \begin{pmatrix} x_N \\ y_N \end{pmatrix} \underbrace{\begin{pmatrix} x_{N+1} \\ 0 \end{pmatrix}}_{e_{N+1}^{(0)}}$$

- The update rule for linear attention is

$$e_j^{(1)} = e_j^{(0)} + P[W_V E_0][E_0^T \Phi e_j^{(0)}]$$

$$\begin{aligned} [W_V E_0][E_0^T \Phi e_j^{(0)}] &= \begin{pmatrix} 0_{d\times 1} & 0_{d\times 1} & \cdots & 0_{d\times 1} & 0_{d\times 1} \\ y_1 & y_2 & \cdots & y_N & 0 \end{pmatrix} \begin{pmatrix} x_1^T x_j \\ x_2^T x_j \\ \vdots \\ x_{N+1}^T x_j \end{pmatrix} \\ &= \sum_{i=1}^{N} \begin{pmatrix} 0_d \\ y_i \end{pmatrix} x_i^T x_j \end{aligned}$$

Try to Understand Layer-1 Attention & Skip Connection

$$\underbrace{\begin{pmatrix} x_1 \\ y_1 \end{pmatrix}}_{e_1^{(0)}} \underbrace{\begin{pmatrix} x_2 \\ y_2 \end{pmatrix}}_{e_2^{(0)}} \quad \cdots \quad \begin{pmatrix} x_N \\ y_N \end{pmatrix} \underbrace{\begin{pmatrix} x_{N+1} \\ 0 \end{pmatrix}}_{e_{N+1}^{(0)}}$$

- The update rule for linear attention is

$$e_j^{(1)} = e_j^{(0)} + P[W_V E_0][E_0^T \Phi e_j^{(0)}]$$

$$\begin{aligned} P[W_V E_0][E_0^T \Phi e_j^{(0)}] &= \begin{pmatrix} 0_{d\times d} & 0_{d\times 1} \\ 0_{1\times d} & -\alpha \end{pmatrix} \sum_{i=1}^{N} \begin{pmatrix} 0_d \\ y_i \end{pmatrix} x_i^T x_j \\ &= -\alpha \sum_{i=1}^{N} \begin{pmatrix} 0_d \\ y_i \end{pmatrix} x_i^T x_j \\ &= -\begin{pmatrix} 0_d \\ \Delta f_j^{(0)} \end{pmatrix} \end{aligned}$$

Try to Understand Layer-1 Attention & Skip Connection

$$\underbrace{\begin{pmatrix} x_1 \\ y_1 \end{pmatrix}}_{e_1^{(0)}} \underbrace{\begin{pmatrix} x_2 \\ y_2 \end{pmatrix}}_{e_2^{(0)}} \quad \cdots \quad \begin{pmatrix} x_N \\ y_N \end{pmatrix} \underbrace{\begin{pmatrix} x_{N+1} \\ 0 \end{pmatrix}}_{e_{N+1}^{(0)}}$$

- The update rule for linear attention is

$$e_j^{(1)} = e_j^{(0)} + P[W_V E_0][E_0^T \Phi e_j^{(0)}]$$

$$\begin{aligned} e_j^{(0)} + P[W_V E_0][E_0^T \Phi e_j^{(0)}] &= \begin{pmatrix} x_j \\ y_j \end{pmatrix} - \begin{pmatrix} 0_d \\ \Delta f_j^{(0)} \end{pmatrix} \\ &= \begin{pmatrix} x_j \\ y_j - \Delta f_j^{(0)} \end{pmatrix} \end{aligned}$$

Summary of Update Process via Linear Attention Layer k to Layer $k+1$

$$\underbrace{\begin{pmatrix} x_j \\ y_j - \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_j) \end{pmatrix}}_{j\text{th input to layer } k+1 \text{ attention}} \implies \underbrace{\begin{pmatrix} 0_{d\times 1} \\ -\Delta f_{\theta_{k+1}}(x_j) \end{pmatrix}}_{j\text{th output from layer } k+1 \text{ attention}}$$

$$\implies \underbrace{\begin{pmatrix} x_j \\ y_j - \sum_{k'=1}^{k+1} \Delta f_{\theta_{k'}}(x_j) \end{pmatrix}}_{\text{skip connection: add input and output}}$$

- The output for the query x_{N+1} is

$$\begin{pmatrix} x_{N+1} \\ -\sum_{k'=1}^{k+1} \Delta f_{\theta_{k'}}(x_{N+1}) \end{pmatrix}$$

- Inner product with $(0_d, -1)^T$ after final layer of Transformer

Summary

- A Transformer with linear attention, with properly designed parameters, can perform gradient descent in its forward pass for linear functions
- The GD is based on a few shots of contextual data, plus a query x_{N+1}
- Important: The setup does not explicitly update the model parameters θ of $f_\theta(x)$
- It updates the functional approximation, layer by layer, in a manner consistent with GD
- Question 1: Are the above designed parameters ϕ for the Transformer optimal?
- Question 2: Linear attention Transformer only works for linear functions. How do we extend for nonlinear functions?

Summary (1/3)

$$\text{Softmax attention: } e_i \quad \rightarrow \quad e_i + \sum_{h=1}^{H} P^{(h)}[W_V^{(h)} E] \cdot \text{softmax}[E^T \Phi^{(h)} e_i]$$

$$\text{Linear attention: } e_i \quad \rightarrow \quad e_i + \sum_{h=1}^{H} P^{(h)}[W_V^{(h)} E] \cdot [E^T \Phi^{(h)} e_i]$$

- Have demonstrated that for linear data models $f(x) = w^T x$ we can design P, W_V and Φ to implement gradient descent for model weights w with linear-attention Transformer
- Each layer of Transformer implements one step of GD
- Given new contextual data $\mathcal{C} = (x_1, w^T x_1, \ldots, x_N, w^T x_N, x_{N+1})$ linear Transformer can predict $w^T x_{N+1}$ on its forward pass

Summary (2/3)

$$\underbrace{\begin{pmatrix} x_j \\ y_j - \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_j) \end{pmatrix}}_{j\text{th input to layer } k+1 \text{ attention}} \implies \underbrace{\begin{pmatrix} 0_{d\times 1} \\ -\Delta f_{\theta_{k+1}}(x_j) \end{pmatrix}}_{j\text{th output from layer } k+1 \text{ attention}}$$

$$\implies \underbrace{\begin{pmatrix} x_j \\ y_j - \sum_{k'=1}^{k+1} \Delta f_{\theta_{k'}}(x_j) \end{pmatrix}}_{\text{skip connection: add input and output}}$$

- Model does not explicitly update the model parameters w_k
- Updates the function $f_{\theta_k}(x_i) = f_{\theta_0}(x_i) + \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_i)$

Summary (3/3)

- Our hand-crafted design of linear Transformer parameters guided by gradient descent (GD)
- If we train the linear Transformer parameters based on multiple contextual examples $\{\mathcal{C}_m\}_{m=1,M}$ it can do better than GD: *preconditioned* GD
- Accounts for statistics of data and model parameters, $p(X)$ and $p(W)$
- Big assumption: linear models, $f(x) = w^T x$
- Now generalize the idea of linear models: kernel machines

Move Beyond Linear Functions: Map to a *Nonlinear* Feature Space

- Map $x \in \mathbb{R}^d$ to features $\phi(x)$
- Dimension of output from $\phi(x)$ is d', which in general different than d
- Dimension d' could even be infinite
- Perform linear regression in the space of features $\phi(x)$

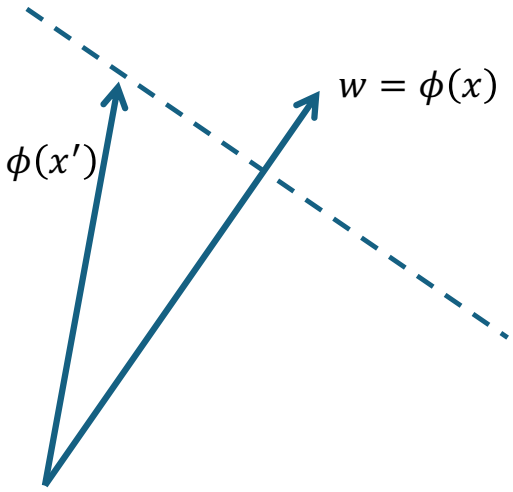
Properties

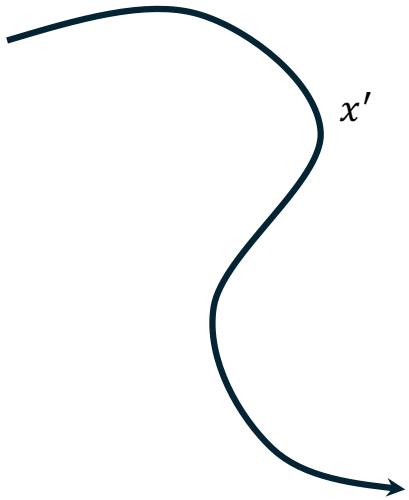
$$f_\theta(x) = w^T \phi(x) + b \quad , \qquad \theta = (w, b)$$

- All $\phi(x)$ along a line perpendicular to w yield the same value for $f_\theta(x)$
- But, if we look at the values of x that manifest $\phi(x)$ along such a perpendicular, typically nonlinear
- In the space of x, surfaces that yield constant functional values are nonlinear
- Key idea: Build nonlinear function $f_\theta(x)$ via nonlinear $x \to \phi(x)$, followed by linear $\phi(x) \to f_\theta(x) = w^T \phi(x) + b$

Nonlinear Features $\phi(x)$ & Linear Model $w^T\phi(x)$

$\phi(x')$ that yields constant $f_\theta(x') = w^T\phi(x') + b$





Values of x' that place $\phi(x')$ on dashed line

Inner Products in Feature Space

- The inner product between $\phi(x_i)$ and $\phi(x_j)$ is
$$\kappa(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$
- The expression $\kappa(x_i, x_j)$ is called a Mercer kernel
- We may define a vector in feature space as
$$w = \sum_{m=1}^{M} \alpha_m \phi(x_m)$$
where $\alpha_m \in \mathbb{R}$ and $x_m \in \mathbb{R}^d$
- The function $f(x) = \phi(x)^T w$ may be expressed
$$f_\theta(x) = \sum_{m=1}^{M} \alpha_m \kappa(x, x_m)$$

Kernel Machines

- Assume that the underlying function $f(x)$ responsible for the data is nonlinear
- Select a kernel type $\kappa(x_i, x_j)$
- Seek to fit data to learn a model of the form

$$f_\theta(x) = \sum_{m=1}^{M} \alpha_m \kappa(x, x_m) + b$$

- Do not explicitly define feature mapping $\phi(x)$, work directly with the corresponding kernel

Example Mercer Kernels

- Linear: $\kappa(x_i, x_j) = x_i^T x_j$
- Radial basis function (RBF): $\kappa(x_i, x_j) = \exp\left(\frac{-\|x_i - x_j\|_2^2}{\sigma^2}\right)$
- Laplacian: $\kappa(x_i, x_j) = \exp\left(\frac{-\|x_i - x_j\|_1}{\sigma^2}\right)$
- Sigmoid: $\kappa(x_i, x_j) = \tanh(\gamma x_i^T x_j + c)$
- Polynomial: $\kappa(x_i, x_j) = (\gamma x_i^T x_j + c)^d$
- Cosine: $\kappa(x_i, x_j) = \frac{x_i^T x_j}{\|x_i\|_2 \|x_j\|_2}$
- Chi-squared: $\kappa(x_i, x_j) = \exp\left(-\gamma \sum_{k=1}^{d} \frac{(x_i(k) - x_j(k))^2}{x_i(k) + x_j(k)}\right)$

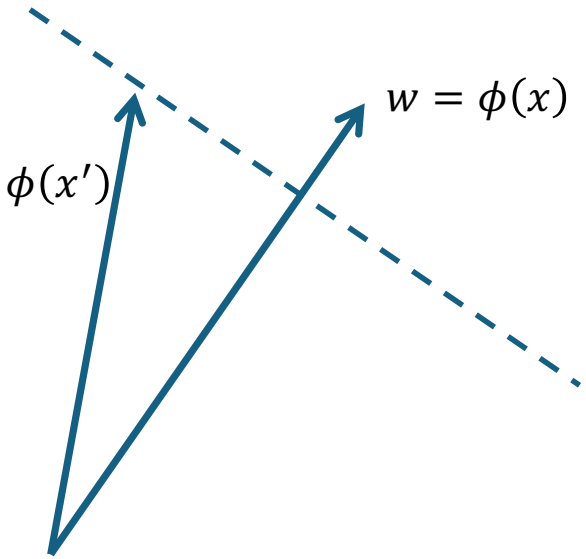
RBF Kernel as an Example

$$\kappa(x', x) = \exp\left(\frac{-\|x - x'\|_2^2}{\sigma^2}\right)$$

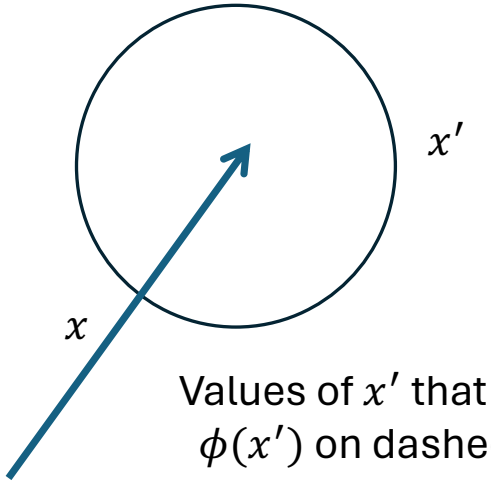
- There is a nonlinear mapping $\phi(x)$ for which $\kappa(x', x) = \phi(x)^T \phi(x')$
- $\phi(x')$ on a hyper-plane perpendicular to $w = \phi(x)$ correspond to positions x' in covariate space on a hypersphere

Radial Basis Function (RBF) Kernel

$\phi(x')$ that yields constant
$f_\theta(x') = w^T\phi(x') + b = \kappa(x', x) + b$



$$\kappa(x', x) = \exp[\frac{-\|x' - x\|_2^2}{\sigma^2}]$$



Values of x' that place $\phi(x')$ on dashed line

RKHS

- The function $\phi(x)$ outputs a d' dimensional vector
- We take inner products in this feature space, in what is termed a Hilbert space
- Inner products in this Hilbert space yield kernel evaluations in the covariate space x
- This is a well studied domain of machine learning, called modeling with a reproducing kernel Hilbert space (RKHS)
- Technical reasons for why termed "reproducing"

Generalization of Few-Shot Learning

- Assume that we are given contextual data

$$\mathcal{C}_m = \{(x_i^{(m)}, y_i^{(m)}\}_{i=1,N}$$

  and a query $x_{N+1}^{(m)}$

- Assume that $y_i^{(m)} = f_{\theta_m}(x_i^{(m)})$, and now $f_{\theta_m}(x)$ is a *nonlinear* function of x

- Posit that the function $f_{\theta_m}(x_i^{(m)})$ can be expressed as a function of kernel $\kappa(x_i, x_j)$, as

$$f_{\theta_m}(x) = \sum_{i=1}^{N} a_i \kappa(x_i^{(m)}, x) + b$$

  where $\theta_m = (a_1, \ldots, a_N, b)$

- Design attention network with skip connection, using attention based on kernel $\kappa(x_i, x_j)$, to infer the latent nonlinear function

Generalize the Game Plan

- Assume that the function $f_\theta(x)$ is in a kernel family with associated kernel $\kappa(x_i, x_j) = \phi(x_i)^T \phi(x_j)$
- Examine how context-dependent gradient descent is performed in such models
- Investigate how the Transformer parameters may be set such that given context (prompt), the Transformer implements kernel-based gradient descent when implementing its forward pass
- Gives insight into how the Transformer can adapt to new prompts/context

Gradient Descent for Linear Model in $\phi(x)$

- Cost function:

$$\mathcal{L}(w) = \frac{1}{2}\sum_{i=1}^{N}[y_i - w^T\phi(x_i)]^2$$

- Gradient descent update:

$$w_{k+1} = w_k + \alpha\sum_{i=1}^{N}[y_i - w_k^T\phi(x_i)]\phi(x_i)$$

- Corresponding function

$$\begin{aligned} f_{w_{k+1}}(x_j) &= w_{k+1}^T\phi(x_j) \\ &= w_k^T\phi(x_j) + \alpha\sum_{i=1}^{N}[y_i - w_k^T\phi(x_i)]\kappa(x_i, x_j) \\ &= f_{w_k}(x_j) + \alpha\sum_{i=1}^{N}[y_i - f_{w_k}(x_i)]\kappa(x_i, x_j) \end{aligned}$$

From Linear Attention to Kernel Attention

- Update rule for nonlinear kernel machine

$$f_{w_{k+1}}(x_j) = f_{w_k}(x_j) + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{w_k}(x_i)] \kappa(x_i, x_j)}_{\Delta f_{w_{k+1}}(x_j)}$$

- Like with linear attention, initialize the weight vector w as all zeros vactor, and therefore $f_{w_0}(x_j) = 0$
- With linear attention, we assumed the model bias b was zero
- Can move beyond that assumption now

Gradient Descent for Linear Model in $\phi(x)$

- Cost function:

$$\mathcal{L}(w, b) = \frac{1}{2} \sum_{i=1}^{N} [y_i - w^T \phi(x_i) - b]^2$$

- Gradient descent update:

$$w_{k+1} = w_k + \alpha \sum_{i=1}^{N} [y_i - w_k^T \phi(x_i)] \phi(x_i)$$

$$b_{k+1} = b_k + \alpha \sum_{i=1}^{N} [y_i - w_k^T \phi(x_i)]$$

- Corresponding function

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] \kappa(x_i, x_j) + \alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)]$$

Two Attention Heads

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] \kappa(x_i, x_j)}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 1 attention}} + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)]}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 2 attention}}$$

- Two attention heads, one with $\kappa(x_i, x_j)$ attention, and the other with constant attention
- Need to return to the form of the Transformer design to see how these two types of attention heads may be manifested in the same model

Transformer Design (1/5)

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] \kappa(x_i, x_j)}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 1 attention}} + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)]}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 2 attention}}$$

- Let's assume that we consider the RBF kernel, $\kappa(x_i, x_j) = \exp\left(\frac{-\|x_i - x_j\|_2^2}{\sigma^2}\right)$ to be specific
- Also assume that the input data are encoded is $e_i = (x_i, y_i)^T$ for $i = 1, \ldots, N$ and $e_{N+1} = (x_{N+1}, 0)^T$

Transformer Design (2/5)

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] \kappa(x_i, x_j)}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 1 attention}} + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)]}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 2 attention}}$$

- For attention head 1, can encode input to RBF-kernel attention via

$$\begin{aligned} W_Q^{(1)} = W_K^{(1)} &= \begin{pmatrix} I_d & 0_{d\times 1} \\ 0_{1\times d} & 0 \end{pmatrix} \\ \kappa(W_K^{(1)} e_i, W_Q^{(1)} e_j) &= \exp\left(\frac{-\| W_K^{(1)} e_i - W_Q^{(1)} e_j \|_2^2}{\sigma^2}\right) \\ &= \exp\left(\frac{-\| x_i - x_j \|_2^2}{\sigma^2}\right) \end{aligned}$$

Transformer Design (3/5)

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] \kappa(x_i, x_j)}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 1 attention}} + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)]}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 2 attention}}$$

- For attention head 2, can encode input to RBF-kernel attention via

$$\begin{aligned} W_Q^{(2)} = W_K^{(2)} &= 0_{(d+1)\times(d+1)} \\ \kappa(W_K^{(2)} e_i, W_Q^{(2)} e_j) &= 1 \end{aligned}$$

Transformer Design (4/5)

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] \kappa(x_i, x_j)}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 1 attention}} + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)]}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 2 attention}}$$

- Key and query matrices for two heads:

$$W_Q^{(1)} = W_K^{(1)} = \begin{pmatrix} I_d & 0_{d\times 1} \\ 0_{1\times d} & 0 \end{pmatrix}, \quad W_Q^{(2)} = W_K^{(2)} = 0_{(d+1)\times(d+1)}$$

Transformer Design (5/5)

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] \kappa(x_i, x_j)}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 1 attention}} + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)]}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 2 attention}}$$

- Key and query matrices for two heads:

$$W_Q^{(1)} = W_K^{(1)} = \begin{pmatrix} I_d & 0_{d\times 1} \\ 0_{1\times d} & 0 \end{pmatrix}, \quad W_Q^{(2)} = W_K^{(2)} = 0_{(d+1)\times(d+1)}$$

- Value matrices, and output matrices

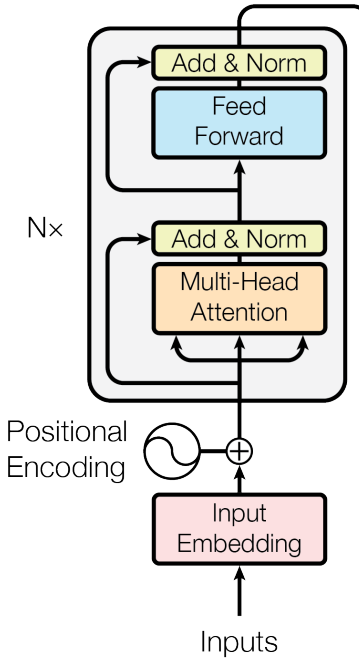
$$W_V^{(1)} = W_V^{(2)} = \begin{pmatrix} 0_{d\times d} & 0_{d\times 1} \\ 0_{1\times d} & 1 \end{pmatrix}, \quad W_O^{(1)} = W_O^{(2)} = \begin{pmatrix} 0_{d\times d} & 0_{d\times 1} \\ 0_{1\times d} & -\alpha \end{pmatrix}$$

- See the power of nonlinear attention and multiple heads

Softmax Kernel & Multiple Attention Heads

- Above construction with kernel regression gives sense of where the nonlinear attention arises from within Transformer
- However, from the perspective of gradient descent, the kernel needs to be Mercer
- The softmax kernel doesn't fit the Mercer kernel perspective, but there are Mercer kernels that are close to it
- We can see the value of multiple head types within the Transformer
- Role of skip connection in attention also clear; needed to implement GD in forward pass

Review the form of the Transformer



- We have explained potential utility of nonlinear (kernel), multi-headed attention
- Allows in-context inference (e.g., gradient descent) in its forward pass
- What about the FF block?